mediumrogue — codebase review (2026-07-19)

A full-tree review across five lenses (game engine, HTTP/security, TS client, UX/feature gaps, duplication). Accessibility excluded per request.

How to read this. Every finding carries **severity** (high/med/low) and **confidence**: `confirmed` = traced in the code (by a reviewer and, for the high-severity ones, re-verified by hand — those are marked ✔verified); `plausible` = a real risk that wants a reproduction before it's certain. `file:line` is where the fault lives. Findings are grouped so each `##` section is a candidate ticket or a cluster of closely-related ones — not by which lens found them, and de-duplicated where several lenses saw the same thing.

Nothing here has been changed. This is a filing document.

⚠ Regressions introduced this session

Two of these came from work merged today; calling them out so they don't get lost among older debt.

- **[high/confirmed ✔verified] Learn button gated on the wrong number.** The panel disables Learn only at `points() === 0`, but a skill now costs 3 (`SkillPointCost`, raised from 1 in #189). A player with 1–2 points sees an enabled button that silently does nothing. One-liner: `disabled={points() < SkillPointCost}` (already importable). `client/src/skills/SkillsPanel.tsx:63` vs `internal/game/skills.go` cost check.
- **[high/confirmed ✔verified] The #170 "⚠ stuck" HUD marker can never appear.** `applyStatus()` is called only on the last line of `onTurn` (`client/src/main.ts:1907`), right after `turnApplied` is set equal to `turnReceived`, so `behind` is always 0 there — and on a mid-handler throw (the exact case it exists for) it's never reached. The crash banner still fires, which is why it shipped unnoticed. Fix: also call `applyStatus()` after stamping `turnReceived` at `main.ts:1529`. (The e2e only asserts the marker *hidden*, so it passed — a partial fix that read as complete.)

Skill state is destroyed on disconnect (data-loss)

- **[high/confirmed ✔verified] The disconnect-sweep archive silently wipes all skill state, then refunds every spent point — an involuntary full respec.** `archiveLocked` captures name/class/species/xp/equipped/backpack but not `learned`, `skillPoints`, `pointsGrantedLevel`, or `activeReadyTurn` (`internal/game/world.go:455-476`), and `restoreArchivedLocked` rebuilds the entity without them (`world.go:764-792`); the disk-archive DTO has the same hole (`snapshot.go:170-177`). A level-5 player with 4 skills who disconnects past the grace and rejoins loses every skill *and* has the bank zeroed — then their next XP event sees `pointsGrantedLevel == 0` with xp intact and re-pays all five levels' points (`world.go:2503-2520`), refunding spent points and breaking the documented "no respec / points are spent once" rule. `#124` added the fields to the live `entityDTO` but not to the archive path — the path *every swept player takes*. `archive_test.go` predates #124 and doesn't cover skills.

The 422 rejection pipeline is built server-side and discarded client-side

The server writes a precise reason into every rejection body (20+ sentinels:

"backpack full", "target out of range", "no path", "can't learn in combat", …).

The client throws all of it away. This is one root cause with several

player-facing symptoms — likely **one ticket** with sub-tasks. Surfaced

independently by the client and UX lenses.

- **[high/confirmed]** ** `postIntent` reduces every response to a boolean and never reads the body** — a 500 is indistinguishable from a typed 422, and no rejection reason can ever reach the player. `client/src/net/session.ts:201-209`. (Chat already reads `ErrorResponse` at `session.ts:329-343` — the pattern exists, it's just not used for intents.)
- **[high/confirmed] A rejected equip/unequip/drop/drink leaves a permanent pending spinner and on-map ⇄ glyph.** The result boolean is voided and `resolvePending` only clears a mark when the item's wire signature *changes*, which a rejection never does — so "unequip with a full backpack" shows an amber spinner forever until an unrelated map click clears it. `client/src/main.ts:1247-1263`, `client/src/gear/store.ts:261-280`.
- **[high/confirmed] A rejected attack leaves the committed crosshair + lit target tiles**, implying "locked in" while the bubble is still waiting on you. `attackAt` / `meleeAt` discard the accept boolean; `walkTo` handles it and attacks don't. This is exactly the #130/#133 stale-target 422 — which the client ignores. `client/src/main.ts:1425`, `:1451`.
- **[medium/confirmed] A network-failed POST escalates into the red "client stopped updating — reload" crash banner.** Intent POSTs are floating promises with no `.catch`; a rejected fetch lands in the global `unhandledrejection` handler, which declares the client dead during any deploy or wifi blip while SSE quietly auto-reconnects. `client/src/main.ts:436`, `:1973`, floating sites at `:1935/:1947/:1250-1262/:1240/:1280`.
- **[medium/confirmed] The pickup modal hardcodes "backpack full" for any rejection** — lose a pickup race and you're told your backpack is full. `client/src/gear/PickupModal.tsx:48`.
- **[low/confirmed] A rejected Learn (e.g. in combat) is silent** — the button isn't disabled in combat and nothing shows. `client/src/main.ts:1240`.

Game-logic / determinism faults (engine)

- **[med-high/confirmed] A player who dies in the same bubble turn as a kill still gets the kill XP.** The award loop checks `e.hp > 0` (`world.go:1528`) *after* `resolveCombatLocked` has already respawned the dead player at full HP — so the "not surviving, earns nothing" guard can no longer be false. Mutual kill: XP floors to level start, then gains the award on top. The pinning test `TestPlayerDyingSameTurnAsMonsterGetsNoKillXP` (`xp_test.go:303`) is a false pin — it uses `ResolveCombatOnlyForTest`, which never runs the award loop. **Wants a reproduction test to confirm the live path.**
- **[medium/confirmed] Entity-targeted ranged attacks can hit victims outside the resolving domain.** `queueAttackLocked`'s invariant ("anything a ranged attack can reach is already in the shooter's bubble", `world.go:1066-1072`) has been false since #95 made bubble edges sight-gated while ranged attacks stayed distance-only. `resolveEntityTargetedLocked` fetches its victim from `w.entities`, not the domain's `byHex` (`world.go:2244`). Consequences: (a) shoot a monster through a rock every world tick — no bubble forms, its aggro raycast is blocked so it never responds, it dies in the world domain (kill XP dropped) but `dropLootLocked` still runs → **risk-free loot farming through walls**; (b) snipe a monster inside someone else's frozen bubble at world cadence. The invariant comment itself says to add an in-domain guard the day it's violated; #95 violated it and none was added.
- **[medium/confirmed ✔verified] Blink ignores occupancy at submit and resolution** (from #161, this session). `useSkillLocked` checks range, walkability, LOS only; `resolveActivesLocked` teleports with no `blockedFor` check — `byHex[target] = append(...); e.hex = target` (`world.go:1957-1960`), while every ordinary mover is gated by opposing-held/StackCap rules. Blink onto a melee monster's hex → co-occupancy where the monster's `Pathfind(from==to)` is empty and it can never attack: **a permanent safe spot**. Blink onto a 5-stack breaks `StackCap`. `blink_test.go` has no occupied-destination case.
- **[low/confirmed]** ** `SetLogger` writes `w.logger` without the mutex** (`world.go:538-544`) while its mirror `SetAnnounce` locks and every reader reads under the lock. Safe only because it's called before `Run`; any future log-level reload is a data race. Lock it or document the before-`Run` contract.

Security (proportionate to a ~15-friend trusted group)

- **[medium/confirmed ✔verified]** **A player can name themselves `system` and impersonate server announcements** (party ops etc.), or clone another player's name and speak as them. `validName` checks only rune count ≤ 24; the client styles any `sender === "system"` as a server line. Reject `system` (case-insensitively) and duplicate names at join. `internal/game/world.go` `validName`, `internal/server/chat.go:60`, `client/src/chat/ChatPanel.tsx:35`.
- **[medium/confirmed] No cap on concurrent SSE streams, and no token needed to open one.** Each `GET /api/events` pins a goroutine and pays a full per-viewer `SnapshotFor` (built under `w.mu`) + `json.Marshal` every tick. A few thousand EventSources take the world lock thousands of times per turn, stalling resolution for everyone — CPU/memory exhaustion, zero credentials. A per-IP or global cap (or `netutil.LimitListener`) closes it. `internal/server/events.go:27-100`.
- **[medium/confirmed ✔verified (with correction)] Unauthenticated join spam grows state without bound.** Empty-token `POST /api/join` mints an entity per call, no rate limit or player cap; swept entities are archived by token and pruned *only on re-join with that exact token* (`world.go:786`). Spamming fresh random tokens means those archive entries never re-join, so they persist forever — into every snapshot. (The agent's "never pruned" is precise as "never pruned for abandoned tokens".) `internal/server/api.go:14`, `internal/game/world.go:670`.

- **[low/confirmed] The full bearer token rides the SSE URL query string** (`events.go:57`) — behind SWAG/nginx its access logs record query strings, so the 128-bit secret lands in proxy logs in full, against the repo's own 8-char-prefix logging discipline. Move to a cookie or a POST-then-stream ticket, or at minimum suppress query logging in the deploy config.
- **[low/confirmed] No chat rate limit** — length is capped at 500 runes but a player can loop `POST /api/chat` and flood every panel. A per-token ~1-line/sec bucket fits. `internal/server/chat.go:17-55`.
- **[low/confirmed]** **Only `ReadHeaderTimeout` is set** — `MaxBytesReader` caps body size, not time, so a trickled POST body pins a goroutine indefinitely. `cmd/rogue/app/app.go:122-130`.

UX & information gaps

- **[high/confirmed ✔verified] The start screen's Human card sells a retired perk** — "Learns faster: bonus XP on every award" — but #124 replaced Human's XP bonus with +1 skill point/level. New players choose a species on false information. `client/index.html:1224`.
- **[medium/confirmed] Your own numeric HP is visible nowhere** — the HUD line is `Lv · XP · (q,r)`, the over-dot bar shows only when damaged and carries no number, yet any monster's exact HP is hover-readable. `client/src/main.ts:1650`, `client/src/render/entities.ts:363-375`.
- **[medium/confirmed] Monster threat stats are invisible** — hover shows name + HP only; damage, damage *type* (the entire point of the #92 resist arc — nothing warns you a dragon deals fire), and reach (a Kin Archer shoots from 3 hexes with no on-map cue) are all server-side only. `client/src/main.ts:2000-2022`.
- **[medium/confirmed] No level-up or "points earned" moment** — the server announces kills and deaths but never a level; the bank shows only inside the `k` panel, so a player who never presses `k` never learns points exist.
- **[medium/confirmed] Active-skill cooldown/range/ready never reach the wire.** `SkillView` has no active fields; a learned Blink shows a flavor line and no numbers, with no way to know its range or cooldown until a submit bounces. This is a **prerequisite for #161's client half** — that slice needs protocol work, not just UI. `protocol.go:485-497`, `skills.go:509-531`.
- **[medium/confirmed]** **Movement keys, SPACE-wait, and `k` appear nowhere on screen** and the start screen teaches no controls. `client/src/input/keys.ts:8-23`.
- **[medium/confirmed] Parties are chat-command-only and unteachable in-game** — no on-screen mention of `/invite` / `/accept` / `/leave`, and no `/help` verb; an unknown command just errors. `internal/server/chat.go:78-112`.
- **[medium/confirmed] Escape closes only the character panel** — not the skills panel (which also has no × button) and not the pickup modal. `client/src/main.ts:1956-1957`.
- **[medium/confirmed] Death is a silent teleport** — the only feedback is a scrolling chat line; no first-person death/respawn banner, and the XP loss to the level floor is never surfaced. `internal/game/world.go:2585-2600`.
- **[low/confirmed] A rejected pickup stays disabled until you leave the hex** even after freeing space — `rejected` clears only on `moved`. `client/src/gear/store.ts:313-341`.
- **[low/confirmed]** Chat typed before join vanishes with no system line (`chat/store.ts:38-41`); world-reset dumps a returning player to the start screen with no explanation (`main.ts:1532-1537`); server-down-at-load shows only a tiny dim status line (`main.ts:2031-2033`).

- **[low/plausible]** The enemy hover tooltip goes stale under a stationary cursor — recomputed only on `pointermove`, so a monster that dies/moves leaves a lingering tooltip. `client/src/main.ts:1976-2023`.

Docs / convention drift

- **[medium/confirmed] FEATURES.md contradicts the code and itself on skill points** — §4's constants table and the XP section say `SkillPointsPerLevel = 2` (now 3), and `SkillPointCost` is missing from §4 entirely, while §1 correctly says 3. Violates "values come from code, never memory". `docs/FEATURES.md:966`, `:332`.
- **[low/confirmed]** **QuestPanel uses `<For>` over per-bundle lists** — the recurring remount trap CLAUDE.md names by example ("quest rows"). Latent because quest rows have no buttons yet; the first "take" button will eat clicks (the CI e2e click-timeout bug). `client/src/quest/QuestPanel.tsx:43`, `:60`. (StatTooltip:119 is a cosmetic second instance.)
- **[low/confirmed]** `GameDebug.skillsPanelOpen`'s doc comment claims "the S key / HUD toggle"; the key is `k` and no HUD toggle exists. `main.ts:141`.

Client bugs (lower severity)

- **[low/confirmed]** `reconnect()` leaves the previous stream's watchdog armed; it can fire mid-handshake and close the new connection before `open` — one spurious reconnect cycle after a rejoin under load; self-heals. `client/src/net/events.ts:130-136`.
- **[low/confirmed]** `window.game.name` desyncs for a returning player — set to the default "traveler" on reclaim until the first bundle carries the real name. `client/src/main.ts:1141`.
- **[low/confirmed]** The `pointermove` tooltip scan runs a `positions.find` + style writes on every pixel event, outside `setHoveredHex`'s hex-change guard — the one per-pixel O(n) in the client, trivially moved behind the guard. `client/src/main.ts:2000-2022`.

Server correctness / performance (lower severity)

- **[low/confirmed]** ** `writeTurn` builds the whole per-viewer bundle under `w.mu` just to compare `Turn == lastSent` and discard it** — paid on every coalesced no-op wake and every current-watermark reconnect. A `World.Turn()` accessor checked before `SnapshotFor` avoids it. `events.go:119-124`.
- **[low/confirmed]** ** `GET /api/map` re-marshals the immutable ~1,800-tile map every request** and is unauthenticated — marshal once at startup, serve cached bytes. `routes.go:39-43`.
- **[low/plausible]** SSE responses lack `X-Accel-Buffering: no`; behind nginx with default `proxy_buffering on`, turn frames can arrive in bursts and feel frozen while "healthy". `events.go:36-38`.
- **[low/confirmed]** Oversized bodies return 400 "malformed JSON" instead of 413, and trailing garbage after the first JSON value is accepted (no second `Decode`/EOF check). `json.go:24-28`.
- **[low/plausible]** A successful `SubmitIntent` doesn't refresh the disconnect grace — only open streams do — so a player whose proxy reaped their SSE but who keeps clicking can be swept mid-play. `world.go:926-964`.

Duplication & simplification (cleanup tickets)

Every count below was spot-checked.

e2e (no shared helpers file exists — 28 specs each redeclare everything):

- **[high/confirmed ✔verified]** The `declare global { interface Window { game: GameDebug } }` block is copied in **27 of 28** specs → one ambient `client/e2e/global.d.ts`.
- **[high/confirmed]** The "find nearest monster, step toward it via combatMoves" chase block is copied ~8× → `client/e2e/helpers.ts` exporting `chaseIntoCombat` (combat.spec.ts:36 & :142, ranged.spec.ts:55, attack-highlight.spec.ts:37, melee-feedback.spec.ts:57, layout.spec.ts:35, autowalk.spec.ts:55).
- **[high/confirmed]** Hex math is hand-rolled per spec although `client/src/render/hex.ts` exports `hexDistance` / `DIRECTIONS` / `neighbor` and is importable from specs — ~15 top-level and in-evaluate copies + byte-identical `pickDistance2Destination` in walk.spec.ts:37 & procgen.spec.ts:38.
- **[medium/confirmed]** Identity-seeding + goto-and-wait preamble repeated across the suite → `seedIdentity(page, opts)` + `gotoReady(page)`; also duplicate `TURN_GATED` (inventory.spec.ts:48, client-alive.spec.ts:23) and two `seedRogue`s.

Go integration tests:

- **[medium/confirmed]** World-boot tail (chatBroker → world.Run → server.New → httptest.NewServer → Cleanup) copied 8× → `serveWorld(t, …)` in testmain_test.go.
- **[medium/confirmed]** Four join helpers repeat POST+status+decode → one `joinWith(t, ts, req)`. Two SSE turn-frame decoders byte-identical → one `decodeTurnFrame`.

Go server (internal/):

- **[medium/confirmed ✔verified]** The `slices.SortFunc(x, func(a,b *entity) int { return int(a.id-b.id) })` closure appears **12×** → one `byEntityID` comparator.
- **[medium/confirmed]** `export_test.go` repeats a "lock, lookup, mutate one field" wrapper ~16× → `withEntityForTest(id, fn)`.
- **[low/confirmed]** Token-auth guard copied 5× across party.go/quest.go → `playerByTokenLocked`. The 8-field itemDef→wire projection duplicated between the ground-item view and `itemViewOf`. `NewWorld`'s 7 positional args re-typed at ~17 sites → a `WorldConfig` struct or shared `newTestWorld`.

Client (client/src/):

- **[medium/confirmed]** `main.ts` is a 2033-line god file; concrete extractable seams (in dependency order): the `GameDebug` / `window.game` block (~570 lines, no logic) → `debug-surface.ts`; start-screen/identity flow (440-660) → `ui/startScreen.ts`; combat-reach/highlight (832-1050) → `tactics.ts`; the 380-line `onTurn` body is the real monolith once those are out.
- **[medium/confirmed]** `isRangedAttackClick` (main.ts:736) and `attackTilesFor` (main.ts:911) independently re-implement the per-weapon range/AoE/hostile rules and must agree tile-for-tile — a future divergence shows a highlight the click won't honor. One resolver should feed both.
- **[low/confirmed]** The three hex-overlay Pixi layers re-implement the same tile draw → `drawHexTile` in render/hex.ts (attack.ts:50, range.ts:48, hover.ts:46). `mustGet` duplicated (timer.ts:57 vs main.ts:389). Pickup-modal mirror mapping copy-pasted (main.ts:638 vs :1748).

Explicitly checked and NOT flagged (deliberate decisions)

So a future reviewer doesn't re-open them: disk-vs-wire DTO separation (commented), the three-places-must-agree vocabulary (commented), the client/server `hexDistance` mirror and integration `combat_test.go`'s independent copy (documented deliberate), token-in-body auth and non-constant-time compare (proportionate at this threat model), origin-guard DNS-rebinding limits (documented), hub coalescing drops (documented contract), snapshot-per-viewer cost (documented), and the map-iteration → rng sites, which consistently sort first (determinism discipline is applied cleanly throughout).